

Temperature Cabinet Setpoint Control from CODESYS HMI — Research & Deliverable Package

TL;DR

- All five research areas are resolved against primary sources: the Watlow F4S natively exposes EIA-232 and EIA-485 Modbus RTU in its base unit at 9600 or 19200 baud, register 100 = Input 1 Value (actual temp) and register 300 = Set Point 1 (static), both with one implied decimal — so your confirmed comms map and the 9600 / addr-1 / 8N1 settings are correct, and the edge-triggered FC06 / cyclic FC03 architecture is exactly right.
- One stated project assumption must be corrected: register 300 is **not** documented by Watlow as EEPROM-backed. The F4S/D spec sheet describes data retention via **battery-backed RAM (7-year)**, and the "cyclic writes damage EEPROM" caution belongs to Watlow's **SD31**, a different product ("Writing to registers continuously, such as ramping set points via comms, will damage the SD31 EEPROM memory"). Keep edge-triggered writing — it is correct engineering — but restate the rationale as "write once per operator action; the F4 follows reg 300 only in static mode," not "prevent EEPROM wear." (Watlow) (Watlow)
- The deliverables below (ST POU's, WebVisu spec, runnable HTML mockup, deploy/test plan, tooling) are built to your confirmed hardware and to verified CODESYS/Watlow behaviour, with every place the real behaviour diverges from a naive implementation flagged inline.

Key Findings

1. Watlow F4S is correctly characterized — with two important nuances

- Native comms confirmed verbatim by the Watlow F4S/D spec sheet: "EIA-232 and EIA-485 serial communication interfaces are included in the base unit of both the SERIES F4S (single channel) and the SERIES F4D (dual channel) controllers. The baud rate is selectable as either 9600 or 19200 Kbaud. The protocol is Modbus RTU." No add-on comms module is needed; 9600/8N1 is a valid native setting, and the null-modem wiring is consistent with TestEquity's guidance that the chamber's 9-pin D-Sub is "wired to accommodate an RS-232 null-modem cable." (Watlow) (Testequity)
- Register map confirmed by TestEquity (an F4 chamber vendor): "The actual chamber temperature reading is Modbus register 100 (Input 1 Value)... The static temperature set point is Modbus register 300 (Set Point 1). The temperature set point during a profile is Modbus register 4122 (Set Point 1, Current Profile Status). The decimal points are implied. For example, 1005 is actually 100.5 and -230 is -23.0." This validates reg 100, reg 300, and the $\times 10$ scaling exactly. (National Instruments)
- **NUANCE A (mode ownership):** Register 300 owns the setpoint **only while the F4 is in static mode** (no profile running). If a ramp/soak profile is running, the controller follows

the profile step setpoint, and register 4122 ("Set Point 1, Current Profile Status") is READ-ONLY. A write to 300 during a running profile will "succeed" on the bus but not change what the chamber does. The FB therefore treats "confirmed" as "reg 300 read-back equals what I wrote" — which is safe — but the operator must understand supervisory writes take effect only in static mode.

- **NUANCE B (EEPROM myth):** There is no statement in the Watlow F4/F4S/D manual that register 300 is EEPROM-backed or that cyclic writes damage memory. The spec sheet describes "Data retention upon power failure via nonvolatile memory (7 years for battery backed RAM)." Static/battery-backed RAM does not have a limited write-cycle life the way EEPROM does. Edge-triggered writing is still correct (don't clobber the register every scan and don't fight a running profile), but do not justify it with EEPROM wear.

2. Modbus addressing off-by-one is the single most likely bench-check surprise

Watlow lists "relative" register numbers; the F4 manual says to convert to absolute values, add 40001 (register 100 → 40101, register 300 → 40301). On the Modbus wire, PDU addressing is 0-based, so absolute 40101 → wire offset 100 and 40301 → wire offset 300. CODESYS's channel "Offset" field is the raw 0-based value, so entering **100** and **300** is correct. Because vendors disagree on the ± 1 convention, if the read-back times out at offset 100/300, try **99/299** — this exact off-by-one has burned multiple F4 integrators (Control.com/NI threads). ([Control.com](#))

3. CODESYS Modbus RTU serial master — verified device tree and trigger behaviour

- Device tree order (CODESYS V3.5): right-click device → Add Device → Fieldbuses → Modbus → **Modbus COM** (serial port; baud/parity set here) → under it **Modbus Master, COM Port** (select RTU; Response Timeout; Time between frames) → under it **Modbus Slave, COM Port** (unit/slave address; channels).
- Channels are defined on the Modbus Slave "Modbus Server Channel" tab. Access types include "Read Holding Registers (function code 03)" and "Write Single Register (function code 06)." Trigger options are **CYCLIC**, **RISING_EDGE**, and **Application** (the last via the ([ModbusChannel1](#)) FB). The CODESYS help warns: "Channels triggered by a rising edge can displace channels triggered cyclically" — a real scheduling constraint, so keep channels minimal. ([helpme-codesys](#)) ([helpme-codesys](#))
- The rising-edge trigger variable is mapped on the slave's **I/O Mapping** tab; the write fires on the 0→1 transition of that boolean. This is the mechanism that makes FC06 edge-triggered.
- **Bus-cycle-task gotcha:** the Modbus COM device has a Bus Cycle Task setting. If left "unspecified," or if "Always update variables" is off, mapped variables may not update. The documented fix (CODESYS Forge, incl. WAGO PFC200 threads) is to set **Bus cycle task = Main task** and **Always update variables = Enabled** ("Enabled2 / Always in bus cycle task"). Set both explicitly. ([CODESYS Forge](#))
- **Auto-reconnect gotcha (RTU serial):** users report that after an RTU cable disconnect puts the slave in error, comms do **not** auto-restore. CODESYS Forge: "When I re-connect the cable the slave communication does not start automatically... I need to restart the

RTE"; another: "The only way to restart communication is to do a reset or reset `(Modbus_Slave_COM_Port.xError)` bit." The FB must actively recover — reset the slave error and re-issue on reconnect — not just wait. `(CODESYS Forge)` `(CODESYS Forge)`

4. Structured Text patterns confirmed, including a classic R_TRIG+TON trap

- `(R_TRIG.Q)` is TRUE for exactly one scan on a rising edge (CODESYS/Beckhoff standard) — ideal for one-shot FC06 triggering. `(Beckhoff)`
- **TRAP:** Do **not** drive a TON watchdog's `(IN)` with an `(R_TRIG.Q)` pulse — the pulse drops after one scan and the timer resets before PT elapses. Drive `(TON.IN)` with a **level** signal that stays TRUE for the whole window being timed. This is a widely documented CODESYS beginner failure. `(Industrialmonitordirect)`
- Enum + CASE state machines with an explicit ELSE branch (safe default) and edge-detected command inputs are the accepted IEC 61131-3 pattern.

5. WebVisu conventions confirmed, with three real gotchas

- **Logo/background:** images live in an Image Pool and are referenced by a static Image ID on an Image element; a full-page background is set in Visualization Manager → Advanced Settings → Background Configuration (Image ID). `(CODESYS)`
- **Pop-up dialog:** set the pop-up visualization's type to "Dialog" (Properties → Visualization tab → Dialog). On the calling button, Input configuration → OnMouseClicked → **Open dialog** = target dialog (option "Open dialog modal", optionally centered). Close it with a **Close dialog** action inside the dialog (with a Result like OK/Cancel). Dialog interface variables are exposed automatically as `<DialogName>_VISU_STRUCT`. `(CODESYS)` `(CODESYS)`
- **Numeric input + min/max:** bind to the variable, use a formatting placeholder. For a value with one decimal use `(%.1f)` and a **REAL** variable — if you use an integer type with a decimal placeholder the field goes red as "out of range" (documented CODESYS Forge issue). Min/Max limits configured in the input action turn the field red when violated. `(CODESYS Forge)`
- **Colour from a boolean:** use the element's dynamic Color variables property — "The Element property is used as an interface for project variables to dynamically control colors at runtime." Drive fill colour from the status boolean. `(Codesys)`
- **Publish:** add a WebVisu object under the Visualization Manager (Add Object → WebVisu Object), name the `(.htm)` start page, optionally "Use as default page"; served by the runtime web server (default port 8080) at `(http://<ip>:8080/<name>.htm)`. GOTCHAS: (a) WebVisu needs an HTML5-canvas browser; (b) a dialog background can render **transparent** in WebVisu if not explicitly filled; (c) dynamic images are aggressively browser-cached — fine for a static logo, but don't rely on hot-swapping a same-named image file. `(CODESYS + 2)`

6. Claude Code / Claude Design tooling is mature and directly applicable

- MCP is the integration standard; the **official GitHub MCP server** (`(github/github-mcp-`

`server`) connects Claude Code to the repo for issues/PRs/Actions. The npm `@modelcontextprotocol/server-github` is **deprecated** — its npm page states "Package no longer supported... Development for this project has been moved to github.com/github/github-mcp-server." Use the official Go/Docker/remote server. `npm`

- Custom slash commands have been merged into **Skills** (`.claude/skills/<name>/SKILL.md`); `.claude/commands/` still works as legacy. **Subagents** (`.claude/agents/`) give isolated context for research/review. **Plugins** bundle skills + agents + hooks + MCP. `Substack`
- **Claude Design** (Anthropic Labs, launched **April 17, 2026**, powered by **Claude Opus 4.7**, per Anthropic's announcement: "Claude Design is powered by our most capable vision model, Claude Opus 4.7, and is available in research preview for Claude Pro, Max, Team, and Enterprise subscribers") generates interactive HTML/CSS/JS prototypes from natural language and can hand off to Claude Code — a strong fit for iterating the WebVisu mockup before committing layout to CODESYS. It is front-end only and export-oriented (HTML/PDF/PPTX/Canva). `Anthropic`

Details

A. CODESYS Structured Text POU (production-ready)

All comms/scaling/limits are configurable FB inputs or GVL constants, defaulting to the confirmed values (9600, addr 1, 8N1, $\times 10$ scaling, 3 s watchdog, 30-130 °C).

DUT `E_SetpointState`

```
TYPE E_SetpointState :  
(  
    IDLE      := 0,    // No request pending; setpoint confirmed/steady  
    READY     := 10,   // Validated request staged, ready to write  
    WRITING   := 20,   // FC06 edge-write issued, waiting for channel done  
    CONFIRM   := 30,   // Comparing FC03 read-back against target  
    FAULTED   := 99    // Latched fault; requires reset  
);  
END_TYPE
```

DUT `E_FaultCode`

```

TYPE E_FaultCode :
(
    NO_FAULT      := 0,
    COMMS_TIMEOUT := 1,    // No valid FC03 response within watchdog
    WRITE_FAILED  := 2,    // FC06 channel reported error
    NOT_ACCEPTED  := 3,    // Read-back != target beyond tolerance
    RANGE_LOW     := 4,    // Request below rMinSetpoint
    RANGE_HIGH    := 5,    // Request above rMaxSetpoint
    OVER_TEMP     := 6     // Spare EL1409 DI over-temp interlock
);
END_TYPE

```

GVL **GVL_Modbus** — channel-mapped raw variables (wire to the slave I/O Mapping tab)

```

{attribute 'qualified_only'}
VAR_GLOBAL
    // FC03 cyclic read, offset 100 (fallback 99), length 1 -> actual temp raw
    (x10)
    wInput1Value      : WORD;
    // FC03 read-back, offset 300 (fallback 299), length 1 -> SP1 raw read (x10)
    wSetpoint1Read    : WORD;
    // FC06 write single register, offset 300 -> SP1 raw write (x10)
    wSetpoint1Write   : WORD;
    // Rising-edge trigger var for the FC06 channel (I/O Mapping tab)
    xWriteTrigger     : BOOL;
    // Master/slave diagnostic bits from the device IEC objects
    xModbusError      : BOOL;    // Modbus_Slave_COM.xError
    xModbusDone       : BOOL;    // last transaction done
END_VAR

```

GVL **GVL_HMI** — engineering-unit / operator-facing

```

{attribute 'qualified_only'}
VAR_GLOBAL
    rChamberTemp      : REAL;        // actual temp, deg C (read/10)
    rConfirmedSetpoint : REAL;        // last confirmed SP, deg C
    rReqSetpoint       : REAL := 75.0; // operator entry (dialog), deg C
    xSetBtn            : BOOL;        // dialog SET button
    xResetBtn          : BOOL;        // dialog/banner RESET button
    xSetpointConfirmed : BOOL;        // green banner driver
    xFaultActive       : BOOL;        // red banner driver
    eFaultCode         : E_FaultCode;
    sStatusText        : STRING(80);
END_VAR

```

FB **FB_CabinetSetpointControl**

FUNCTION_BLOCK FB_CabinetSetpointControl

VAR_INPUT

```
    rRequestedSetpoint : REAL;           // deg C from HMI
    xSetCommand         : BOOL;          // HMI SET (edge)
    xReset              : BOOL;          // fault reset (edge)
    xOverTempDI         : BOOL;          // spare EL1409 DI, TRUE = over-
temp
```

// --- configurable, default to confirmed values ---

```
    rMinSetpoint       : REAL := 30.0;
    rMaxSetpoint       : REAL := 130.0;
    rScale              : REAL := 10.0;    // eng -> raw multiplier
    rTolerance          : REAL := 0.1;     // deg C read-back tolerance
    tCommsTimeout      : TIME := T#3S;     // watchdog
```

END_VAR

VAR_OUTPUT

```
    eState              : E_SetpointState;
    eFault              : E_FaultCode;
    xConfirmed          : BOOL;
    xFault              : BOOL;
    rActualTemp         : REAL;
    rConfirmedSetpoint  : REAL;
```

END_VAR

VAR

```
    rtrigSet            : R_TRIG;
    rtrigReset          : R_TRIG;
    tonWatchdog         : TON;
    tonConfirmWait      : TON;
    wTargetRaw          : WORD;
    rReadBack           : REAL;
```

END_VAR

// ---- Edge detect ----

```
rtrigSet(CLK := xSetCommand);
```

```
rtrigReset(CLK := xReset);
```

// ---- Always-live scaling of the cyclic read ----

```
rActualTemp := WORD_TO_REAL(GVL_Modbus.wInput1Value) / rScale;
```

```
rReadBack := WORD_TO_REAL(GVL_Modbus.wSetpoint1Read) / rScale;
```

```
GVL_HMI.rChamberTemp := rActualTemp;
```

// ---- Comms watchdog: LEVEL signal, NOT an edge pulse (avoids R_TRIG->TON trap) ----

```
tonWatchdog(IN := NOT GVL_Modbus.xModbusDone, PT := tCommsTimeout);
```

// ---- Over-temp interlock (highest priority latch) ----

```
IF xOverTempDI THEN
```

```
    eFault := E_FaultCode.OVER_TEMP;
```

```
    eState := E_SetpointState.FAULTED;
```

```
END_IF
```

// ---- Comms loss latch (any state) ----

```

IF tonWatchdog.Q OR GVL_Modbus.xModbusError THEN
    eFault := E_FaultCode.COMMS_TIMEOUT;
    eState := E_SetpointState.FAULTED;
END_IF
CASE eState OF
    E_SetpointState.IDLE:
        xConfirmed := TRUE;
        GVL_Modbus.xWriteTrigger := FALSE;
        IF rtrigSet.Q THEN
            eState := E_SetpointState.READY;
        END_IF
    E_SetpointState.READY:
        xConfirmed := FALSE;
        IF rRequestedSetpoint < rMinSetpoint THEN
            eFault := E_FaultCode.RANGE_LOW;
            eState := E_SetpointState.FAULTED;
        ELSIF rRequestedSetpoint > rMaxSetpoint THEN
            eFault := E_FaultCode.RANGE_HIGH;
            eState := E_SetpointState.FAULTED;
        ELSE
            wTargetRaw := REAL_TO_WORD(rRequestedSetpoint * rScale);
            GVL_Modbus.wSetpoint1Write := wTargetRaw;
            eState := E_SetpointState.WRITING;
        END_IF
    E_SetpointState.WRITING:
        GVL_Modbus.xWriteTrigger := TRUE;    // one-shot rising edge fires FC06
        eState := E_SetpointState.CONFIRM;
    E_SetpointState.CONFIRM:
        GVL_Modbus.xWriteTrigger := FALSE;    // drop trigger to re-arm the edge
        tonConfirmWait(IN := TRUE, PT := tCommsTimeout);
        IF ABS(rReadBack - rRequestedSetpoint) <= rTolerance THEN
            rConfirmedSetpoint := rReadBack;
            GVL_HMI.rConfirmedSetpoint := rReadBack;
            tonConfirmWait(IN := FALSE);
            eState := E_SetpointState.IDLE;
        ELSIF tonConfirmWait.Q THEN
            eFault := E_FaultCode.NOT_ACCEPTED;
            eState := E_SetpointState.FAULTED;
        END_IF
    E_SetpointState.FAULTED:
        xConfirmed := FALSE;
        GVL_Modbus.xWriteTrigger := FALSE;
        // Latched – clears only on reset edge AND only if condition is gone
        IF rtrigReset.Q AND NOT xOverTempDI
            AND NOT tonWatchdog.Q AND NOT GVL_Modbus.xModbusError THEN
            eFault := E_FaultCode.NO_FAULT;
            eState := E_SetpointState.IDLE;
        END_IF
END_CASE

```

```

        END_IF
    END_CASE
    xFault := (eState = E_SetpointState.FAULTED);
    GVL_HMI.xSetpointConfirmed := xConfirmed AND NOT xFault;
    GVL_HMI.xFaultActive      := xFault;
    GVL_HMI.eFaultCode        := eFault;

```

PLC_PRG

```

PROGRAM PLC_PRG
VAR
    fbSetpoint : FB_CabinetSetpointControl;
END_VAR
fbSetpoint(
    rRequestedSetpoint := GVL_HMI.rReqSetpoint,
    xSetCommand        := GVL_HMI.xSetBtn,
    xReset             := GVL_HMI.xResetBtn,
    xOverTempDI        := GVL_Modbus.xModbusError, // REPLACE with mapped
EL1409 DI channel
    rMinSetpoint       := 30.0,
    rMaxSetpoint       := 130.0
);
// Consume dialog button one-shots
GVL_HMI.xSetBtn := FALSE;
GVL_HMI.xResetBtn := FALSE;

```

Note: map `xOverTempDI` to the real EL1409 digital-input channel in the EtherCAT I/O mapping, not to the Modbus error bit (placeholder above).

B. WebVisu design/layout SPECIFICATION (hand-off)

Canvas: 1280 × 800; one main visualization `visMain` plus one dialog `dlgSetpoint`.

Main screen, element-by-element:

- Logo band** — Image element, top, x=0 y=0 w=1280 h=90, static Image ID `ImagePool.OliverLogo` (PNG placed in an Image Pool). Optional full-canvas background via Visualization Manager → Advanced Settings → Background Configuration.
- Title label** — Text element beside logo: "Temperature Cabinet Setpoint Control".
- Gear button** — Button, ~x=1120 y=20 w=120 h=60, text/gear glyph "Set the cabinet temperature setpoint". Input configuration → OnMouseClicked → **Open dialog** = `dlgSetpoint`, "Open dialog modal" = ON.
- Chamber Temperature tile** — Rectangle + Text `Chamber: %.1f °C` bound to `GVL_HMI.rChamberTemp` (REAL); updates from cyclic FC03.
- Confirmed Setpoint tile** — Rectangle + Text `Setpoint (confirmed): %.1f °C` bound to `GVL_HMI.rConfirmedSetpoint`.

6. **Status banner** — Rectangle spanning the bottom; Text bound to `GVL_HMI.sStatusText`. Dynamic fill via Color variables: `(xFaultActive = TRUE → red); (xSetpointConfirmed = TRUE → green)`; else amber (WRITING/CONFIRM). Implement with the element's color-variable interface (toggle-color properties or an INT color selector driven from `eFaultCode`).

Dialog `dlgSetpoint` (Visualization type = **Dialog**):

- **Numeric entry** — Text field `(%.1f)`, variable `GVL_HMI.rReqSetpoint` (**REAL**), input action Write Variable with **Min=30.0 / Max=130.0** (field turns red outside range). Use the VisuDialogs numpad for touch entry.
- **SET button** — OnMouseClicked → Execute ST code `GVL_HMI.xSetBtn := TRUE;` then **Close dialog** (Result: OK).
- **RESET button** — OnMouseClicked → Execute ST code `GVL_HMI.xResetBtn := TRUE;` then **Close dialog** (Result: Cancel).
- Explicitly set the dialog's background fill (WebVisu renders unfilled dialog backgrounds transparent).

Colour logic: red = `xFaultActive`; green = `xSetpointConfirmed`; amber = transitional. Banner text mirrors `eFaultCode` — e.g., "COMMS TIMEOUT", "SETPOINT NOT ACCEPTED", "OVER-TEMP", "RANGE 30-130 °C", "Confirmed 100.0 °C".

C. Runnable single-file HTML mockup

Save as `hmi-mockup.html` and open in any browser. Logo band, working gear button → modal pop-up, numeric entry with range validation, SET/RESET, simulated live tiles, and banner colour logic (green confirmed / amber writing / red fault).

html

```

<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8"><meta name="viewport" content="width=device-width,initial-sc
<title>Cabinet Setpoint HMI – Mockup</title>
<style>
:root{--bg:#0f1720;--tile:#1b2734;--ink:#e8eef4;--mut:#8aa0b3;--grn:#1f9d55;--red:
*{box-sizing:border-box;font-family:Segoe UI,Roboto,Arial,sans-serif}
body{margin:0;background:var(--bg);color:var(--ink)}
header{display:flex;align-items:center;gap:16px;padding:14px 22px;background:#0b11
.logo{font-weight:800;letter-spacing:.5px;background:linear-gradient(90deg,#3aa0ff
header h1{font-size:16px;font-weight:600;margin:0;color:var(--mut)}
.gear{margin-left:auto;background:#22405f;color:#fff;border:0;padding:12px 16px;bo
.gear:hover{background:#2b527a}
main{display:grid;grid-template-columns:1fr 1fr;gap:18px;padding:22px}
.tile{background:var(--tile);border:1px solid var(--line);border-radius:14px;paddi
.tile .lab{color:var(--mut);font-size:13px;text-transform:uppercase;letter-spacing
.tile .val{font-size:44px;font-weight:800;margin-top:8px}
.banner{grid-column:1/3;border-radius:14px;padding:18px 22px;font-size:18px;font-w
.dlg-back{position:fixed;inset:0;background:rgba(0,0,0,.55);display:none;align-ite
.dlg{background:#16202b;border:1px solid var(--line);border-radius:16px;padding:24
.dlg h2{margin:0 0 14px;font-size:18px}
.dlg input{width:100%;font-size:26px;padding:12px;border-radius:10px;border:1px so
.dlg input.bad{border-color:var(--red);color:var(--red)}
.hint{color:var(--mut);font-size:12px;margin:8px 0 16px}
.row{display:flex;gap:12px}
.row button{flex:1;padding:12px;border:0;border-radius:10px;font-size:15px;cursor:
.set{background:var(--grn)}.rst{background:#42566a}
</style></head>
<body>
<header>
<span class="logo">OLIVER MECHATRONICS</span>
<h1>Temperature Cabinet Setpoint Control</h1>
<button class="gear" onclick="openDlg()">⚙ Set the cabinet temperature setpoint<
</header>
<main>
<div class="tile"><div class="lab">Chamber Temperature</div><div class="val"><sp
<div class="tile"><div class="lab">Setpoint (confirmed)</div><div class="val"><s
<div class="banner" id="banner">SETPOINT CONFIRMED – 75.0 °C</div>
</main>
<div class="dlg-back" id="back">
<div class="dlg">
<h2>Enter cabinet setpoint</h2>
<input id="req" type="number" step="0.1" value="75.0" oninput="chk()">
<div class="hint" id="hint">Valid range 30.0 – 130.0 °C</div>
<div class="row"><button class="set" onclick="doSet()">SET</button>

```

```

    <button class="rst" onclick="doReset()">RESET</button></div>
</div>
</div>
<script>
const MIN=30,MAX=130,TOL=0.1;
let target=75.0, confirmed=75.0, temp=75.0, fault=null;
function openDlg(){document.getElementById('back').style.display='flex';chk();}
function closeDlg(){document.getElementById('back').style.display='none';}
function chk(){const v=parseFloat(document.getElementById('req').value);
    const bad=isNaN(v)||v<MIN||v>MAX;document.getElementById('req').classList.toggle
    document.getElementById('hint').textContent=bad?('Out of range — must be '+MIN+'
function doSet(){if(!chk()){fault='RANGE';render();return;}
    target=parseFloat(document.getElementById('req').value);fault='WRITING';render()
    setTimeout(=>{ if(Math.random()<0.06){fault='NOT_ACCEPTED';}else{confirmed=tar
function doReset(){fault=null;render();closeDlg();}
function render(){document.getElementById('conf').textContent=confirmed.toFixed(1)
    const b=document.getElementById('banner');
    if(fault==='RANGE'){b.style.background='var(--red)';b.textContent='RANGE FAULT —
    else if(fault==='NOT_ACCEPTED'){b.style.background='var(--red)';b.textContent='F
    else if(fault==='COMMS'){b.style.background='var(--red)';b.textContent='FAULT —
    else if(fault==='WRITING'){b.style.background='var(--amb)';b.textContent='Writin
    else{b.style.background='var(--grn)';b.textContent='SETPOINT CONFIRMED — '+confi
setInterval(=>{temp+=(confirmed-temp)*0.08+(Math.random()-0.5)*0.2;
    document.getElementById('temp').textContent=temp.toFixed(1);},1000);
render();
</script>
</body></html>

```

D. Deploy checklist + test plan ("Rebuild, Retest, Requalify, Repeat"; PASS only after two consecutive clean runs)

Deploy checklist:

1. USB-RS232 adapter enumerated as `/dev/ttyUSB0`; add under `[SysCom]` in `CODESYSControl_User.cfg` and map to COM Port 1.
2. F4 Setup → Communications: Modbus RTU, **9600**, address **1**, **8N1** (verify on the unit — this is where a stale 19200 will bite).
3. Confirm null-modem wiring: DB9 TX → terminal 15 (Rx); DB9 RX → terminal 14 (Tx); DB9 GND → terminal 16.
4. Modbus COM: **Bus Cycle Task = MainTask** (not unspecified); **"Always update variables" = Enabled**.
5. FC03 read channel offset **100** (fallback 99); FC06 write channel offset **300** (fallback 299); FC03 read-back offset **300**.
6. Verify master + slave devices turn green; confirm reg-300 read shows **750** (=75.0 °C) before any write.

Test cases (each PASS requires two consecutive clean runs):

1. **Read-back accuracy** — $\text{reg } 100 \div 10$ matches an independent thermometer within $\pm 1^\circ\text{C}$.
PASS: match on two runs.
2. **Setpoint write** — enter 100.0, SET; reg 300 raw becomes 1000. PASS: $\text{raw} = \text{target} \times 10$.
3. **Acceptance confirm** — FC03 read-back = target within tolerance sets `xSetpointConfirmed`; banner green. PASS: confirmed within 3 s.
4. **Range validation** — enter 25.0 and 140.0; FB rejects (RANGE_LOW/HIGH), no bus write issued. PASS: no write + red banner both runs.
5. **Comms-loss fault** — unplug `/dev/ttyUSB0` mid-operation; 3 s watchdog latches COMMS_TIMEOUT, banner red. PASS: fault within 3 s.
6. **Write-fail / not-accepted** — force read-back $\neq$ target (e.g., write during a running profile so 300 is ignored, or point read-back at a wrong offset); FB latches NOT_ACCEPTED. PASS: fault raised, not silently confirmed.
7. **Over-temp via spare EL1409 DI** — assert the DI; FB latches OVER_TEMP regardless of state; write trigger forced FALSE. PASS: latch + inhibit both runs.
8. **Fault reset** — clear the source, press RESET; FB returns to IDLE only when the condition is gone. PASS: no reset while active; clean reset when cleared.
9. **Range sweep 30/100/130 °C** — write each; verify chamber PID drives toward each and read-back confirms. PASS: all three confirm on two full sweeps.

After any code/wiring change: **Rebuild** → **Retest the affected cases** → **Requalify (re-run all 9)** → **Repeat** until two consecutive clean full passes.

E. Claude Code / Claude Design tooling recommendations

- **GitHub MCP server (official)** on `0J4884/Temperature-Cabinet-Setpoint-Control-from-CODESYS-HMI` — manage one issue per test case, open a PR per FB revision, watch Actions. Use the official server, not the deprecated npm package.
- **Filesystem / in-repo Claude Code** — version the ST exports (PLCopenXML), the WebVisu spec, and the HTML mockup; use `/diff` before commits.
- **Project subagent** (`.claude/agents/st-reviewer.md`, read-only tools) to review ST against IEC-61131-3 conventions (edge-detected commands, safe ELSE branch, no R_TRIG→TON gating).
- **Project skill** (`.claude/skills/deploy-check/SKILL.md`) encoding the deploy checklist + 9 test cases so `/deploy-check` regenerates a qualification log each iteration.
- **Context7 MCP** for pulling current CODESYS/library docs into context on demand.
- **Claude Design** to iterate HMI look-and-feel (logo band, tiles, banner colour states) fast, then hand the HTML to Claude Code and translate into the WebVisu placement above. Front-end only — it complements, doesn't replace, the CODESYS work.
- **Frontend-design plugin** for polished, non-generic HTML if the mockup is shown to stakeholders.

Recommendations

1. **Bench-check comms before anything else.** Standardize on 9600/1/8N1, verify the F4's own menu (kill any lingering 19200), and confirm you can read reg 300 = 750 before writing. If reads time out, flip offset to 99/299. This single step de-risks the majority of F4 integrations. Threshold to escalate: if the master won't turn green after cabling + config are verified, snoop the line (e.g., portmon/scope) before touching code.
2. **Ship the FB as written but re-label the EEPROM rationale.** Keep FC06 edge-triggered — it's correct — but in comments/docs state the reason as "write once per operator action; the F4 follows reg 300 only in static mode," not EEPROM-wear protection (that caution is the SD31's).
3. **Make static mode a precondition.** Because a running profile makes reg 300 inert, add an operator note (ideally read the profile run-status register) so the HMI can warn "cabinet is running a profile — supervisory setpoint inactive." If you confirm the chamber is never profiled in service, you can drop this check.
4. **Wire the over-temp DI to a real EL1409 channel** and keep OVER_TEMP as the top-priority latch (already coded). The F4's own PID and Set Point High/Low Limits remain the true safety layer — CODESYS is supervisory only.
5. **Set the Modbus COM bus-cycle task explicitly (MainTask + Always update variables) and add active reconnect** (reset slave `xError` on reconnect) so RTU comms recover without a runtime restart.
6. **Adopt tooling in stages:** GitHub MCP + repo now → `/deploy-check` skill + ST-reviewer subagent next → Claude Design for HMI polish last. Benchmark to escalate: if manual test logging exceeds ~15 min/iteration, build the `/deploy-check` skill.

Caveats

- The FB, WebVisu spec, and mockup are engineered from verified CODESYS/Watlow behaviour but are untested on your specific DLS008 / Pi 5 / F4S (SN 038983) hardware — treat the 9-case plan as the acceptance gate.
- Register offset (± 1) and the reg-300-vs-4122 mode question are the two behaviours most likely to differ from a naive implementation; both are flagged and covered by test cases 2, 3, and 6.
- Vendor/forum sources (TestEquity, NI, Control.com) corroborate the register map and scaling; the primary Watlow F4S/D User's Manual (doc 0600-0032-0000) Chapter 7 is the authority — read it end-to-end before final qualification. Note the manual lists Set Point High/Low Limit registers per analog input; confirm the exact numeric IDs against Chapter 7 if you want the FB's range check to mirror the controller's own limits.
- Claude Design details reflect its April 2026 research-preview state (Opus 4.7, front-end only, weekly token limits); capabilities and limits may change.
- WebVisu on CODESYS Control for Linux ARM64 SL requires an appropriate visualization/WebVisu license on the runtime — confirm the license is present before relying on browser access.

- The F4 returns a Modbus exception response with the function code's high bit set (function + 0x80) on rejected/illegal writes, and silently ignores malformed/bad-CRC frames — so a "no response" is not necessarily a rejection; the watchdog (test 5) is what catches that condition.